Přednáška 8: Úvod do neuronových sítí a hlubokého učení

1. Zasazení do kontextu: Od strojového učení k hlubokému učení

V předchozích přednáškách jsme si ukázali, jak umělá inteligence (AI) transformuje materiálové inženýrství, od urychlení objevování nových materiálů po charakterizaci mikrostruktur a predikci vlastností. Viděli jsme příklady, kde konvoluční neuronové sítě (CNN) analyzují snímky mikrostruktur , grafové neuronové sítě (GNN) předpovídají vlastnosti krystalových struktur a transformery se používají k analýze spektroskopických dat nebo k extrakci znalostí z vědecké literatury. Nyní je čas ponořit se hlouběji a pochopit, co se skrývá pod kapotou těchto výkonných nástrojů. Tato přednáška slouží jako klíčový úvod do světa neuronových sítí (NN) a hlubokého učení (Deep Learning, DL), které tvoří základ těchto pokročilých architektur.

1.1 Hierarchie AI: Vyjasnění terminologie

V oblasti umělé inteligence se často setkáváme s pojmy, které jsou používány zaměnitelně, ačkoliv mají odlišný význam. Pro naši další práci je klíčové si tuto terminologii ujasnit. Vztah mezi těmito koncepty lze nejlépe popsat jako hierarchii vnořených množin, podobně jako u ruských matrjošek.

- **Umělá inteligence (Artificial Intelligence, AI):** Je nejširší a nejstarší koncept. Jedná se o obecnou snahu vytvořit stroje nebo počítačové systémy, které mohou vykonávat úkoly vyžadující lidskou inteligenci, jako je řešení problémů, rozhodování, rozpoznávání řeči nebo porozumění jazyku. AI je největší matrjoška, která v sobě skrývá všechny ostatní.
- **Strojové učení (Machine Learning, ML):** Je podmnožinou AI. Místo explicitního programování pravidel se systémy strojového učení učí vzory přímo z dat. Umožňuje strojům zlepšovat svůj výkon na konkrétním úkolu na základě zkušeností (dat), aniž by byly explicitně naprogramovány pro každý scénář. ML je menší matrjoška uvnitř AI.
- **Hluboké učení (Deep Learning, DL):** Je specializovanou podmnožinou strojového učení. Využívá specifickou architekturu zvanou hluboká neuronová síť, která má mnoho vrstev (odtud název "hluboké"). Tato hloubka umožňuje modelům automaticky se učit velmi složité a hierarchické vzory z rozsáhlých datových sad. DL je ještě menší matrjoška uvnitř ML.
- **Neuronové sítě (Neural Networks, NN):** Jsou základními stavebními kameny hlubokého učení. Jedná se o sadu algoritmů, inspirovaných strukturou lidského mozku, které se snaží rozpoznat základní vztahy v datech. Jsou to ony složité vzory namalované na matrjošce hlubokého učení.

1.2 Klíčový rozdíl: Manuální příprava příznaků vs. automatické učení reprezentací

Nejdůležitějším rozdílem, který odděluje hluboké učení od tradičního strojového učení a který je hlavním důvodem jeho úspěchu, je přístup k datovým příznakům (features).

Tradiční strojové učení: V tradičních ML algoritmech, jako jsou podpůrné vektorové stroje (SVM) nebo náhodné lesy (Random Forests), je úspěch modelu silně závislý na kroku zvaném **manuální příprava příznaků** (manual feature engineering). Tento proces vyžaduje, aby odborník v dané oblasti (např. materiálový vědec) pečlivě vybral a extrahoval z hrubých dat relevantní numerické charakteristiky, které algoritmus použije pro učení.

- **Příklad z materiálové vědy (tradiční přístup):** Představme si úkol klasifikovat mikrostruktury oceli z mikroskopických snímků. Metalurg by musel manuálně změřit a zadat do modelu příznaky jako průměrná velikost zrna, fázový podíl vypočtený pomocí softwaru pro analýzu obrazu, poměr stran precipitátů nebo jejich distribuci. Tento proces je nejen zdlouhavý a náročný na odborné znalosti, ale je také subjektivní a může přehlédnout složité vzory, které nejsou na první pohled zřejmé.

Hluboké učení: Modely hlubokého učení tento přístup obrací. Místo manuální extrakce provádějí **automatickou extrakci příznaků**, známou také jako **učení reprezentací** (representation learning). Síť se učí hierarchii příznaků přímo z hrubých dat.

- **Příklad z materiálové vědy (přístup DL):** Konvoluční neuronová síť (CNN) dostane na vstup přímo pixely snímku mikrostruktury. V prvních vrstvách se může naučit detekovat jednoduché prvky, jako jsou hrany a textury. V hlubších vrstvách tyto jednoduché prvky kombinuje, aby rozpoznala složitější struktury, jako jsou hranice zrn. V nejhlubších vrstvách pak může identifikovat celé fáze (např. ferit, perlit, martenzit). Síť si sama "vymyslí" nejlepší příznaky pro daný úkol.

Tento přechod od manuálního k automatickému učení příznaků není pouhým vylepšením nástrojů; představuje fundamentální posun ve vědeckém pracovním postupu a v roli samotného odborníka. V tradičním paradigmatu je vědec *designérem příznaků*, jehož expertíza je zakódována do jím vytvořených vstupů. V paradigmatu hlubokého učení se vědec stává *designérem architektury* a *kurátorem dat*. Jeho expertíza se nyní uplatňuje při výběru vhodné architektury modelu, která odráží fyzikální podstatu dat (např. CNN pro prostorová data, GNN pro atomární grafy), a při navrhování experimentů nebo simulací, které generují vysoce kvalitní a reprezentativní trénovací data. "Černá skříňka" modelu nutí vědce soustředit se více na kvalitu vstupů a strukturu modelu než na mezilehlé reprezentace příznaků.

Následující tabulka shrnuje klíčové rozdíly mezi těmito dvěma přístupy.

Vlastnost	Tradiční strojové učení	Hluboké učení
Příprava příznaků	Manuální, vyžaduje doménové znalosti	Automatická, učí se z dat
Závislost na datech	Dobrý výkon na malých/středních datasetech	Vyžaduje velké množství dat, výkon škáluje s velikostí datasetu

Vlastnost	Tradiční strojové učení	Hluboké učení
Interpretovatelnost	Obecně interpretovatelné ("bílá skříňka", např. rozhodovací stromy)	Většinou "černá skříňka"
Hardware	Běží na standardních CPU	Vyžaduje specializovaný hardware (GPU/TPU)
Typická data	Strukturovaná, tabulková data	Nestrukturovaná data (obrázky, text, grafy)

2. Anatomie neuronové sítě: Od jednoho neuronu ke složitému mozku

Abychom pochopili, jak fungují hluboké neuronové sítě, musíme začít od jejich nejzákladnějšího stavebního kamene. Postupně si ukážeme, jak se z jednoduchých výpočetních jednotek skládají složité architektury schopné řešit komplexní problémy.

2.1 Biologická inspirace: Zjednodušený model neuronu

Umělé neuronové sítě jsou volně inspirovány strukturou a funkcí biologického mozku. Biologické neurony přijímají signály prostřednictvím dendritů, zpracovávají je v buněčném těle, a pokud je signál dostatečně silný, vyšlou vlastní signál dál po axonu k dalším neuronům. Je důležité zdůraznit, že se jedná o funkční analogii, nikoli o přesnou biologickou simulaci. Cílem není replikovat mozek, ale inspirovat se jeho principy pro vytváření efektivních výpočetních modelů.

2.2 Perceptron: První umělý neuron

Nejjednodušší formou neuronové sítě je **perceptron**, který v roce 1957 vyvinul Frank Rosenblatt. Lze jej považovat za první funkční model umělého neuronu.
Anatomie perceptronu:

- **Vstupy (x):** Číselné hodnoty reprezentující příznaky. Pro každý vstup x_i existuje odpovídající váha.
- **Váhy (w):** Čísla, která určují důležitost (sílu) každého vstupu. Váha w_i je přiřazena vstupu x_i.
- **Práh (Bias, b):** Nastavitelná konstanta, která funguje podobně jako absolutní člen v lineární rovnici (y = ax + b). Geometricky posouvá rozhodovací hranici a dává modelu větší flexibilitu.
- **Vážený součet:** Jádro výpočtu, kde se každý vstup vynásobí svou váhou a výsledky se sečtou spolu s prahem. Matematicky: $z = \sum_i w_i x_i + b$.

- **Aktivační funkce:** Jednoduchá prahová (kroková) funkce, která na základě váženého součtu rozhodne, zda neuron "pálí" (aktivuje se). Pokud vážený součet z překročí určitou mez (typicky 0), výstup je 1; jinak je 0.

Funkcí perceptronu je tedy provádět binární klasifikaci. Učí se najít přímku (v 2D) nebo nadrovinu (ve vyšších dimenzích), která dokáže oddělit data náležející do dvou různých tříd.

2.3 Omezení perceptronu: Problém XOR a lineární separabilita

Navzdory své jednoduchosti má perceptron zásadní omezení: dokáže se naučit pouze **lineárně separabilní** problémy. To jsou problémy, kde lze data dvou tříd dokonale oddělit jednou přímkou. Dokáže například implementovat logické operace jako AND nebo OR.
Problém nastává u operace **XOR** (exclusive OR), která lineárně separabilní není. Nelze nakreslit jednu přímku, která by správně oddělila výstupy TRUE od výstupů FALSE. Toto omezení, slavně zdůrazněné v knize "Perceptrons" od Minskyho a Paperta, vedlo na mnoho let k útlumu výzkumu v oblasti neuronových sítí. Právě toto selhání však motivovalo vývoj složitějších architektur.

2.4 Vícevrstvý perceptron (MLP): Překonání limitů pomocí skrytých vrstev

Řešením problému nelineární separability je **vícevrstvý perceptron (Multi-Layer Perceptron, MLP)**. Jak název napovídá, jedná se o síť složenou z více vrstev perceptronů.
Architektura MLP:

- **Vstupní vrstva (Input Layer):** Přijímá hrubá vstupní data (příznaky). Počet neuronů v této vrstvě odpovídá počtu vstupních příznaků.
- **Skryté vrstvy (Hidden Layers):** Jedna nebo více vrstev neuronů mezi vstupní a výstupní vrstvou. Právě tyto vrstvy jsou klíčem k učení složitých, nelineárních vztahů. Učí se mezilehlé, abstraktnější reprezentace dat. Termín "hluboké" v hlubokém učení odkazuje na přítomnost mnoha skrytých vrstev.
- **Výstupní vrstva (Output Layer):** Produkuje finální predikci modelu. Počet neuronů závisí na úloze (např. jeden pro regresi, N neuronů pro klasifikaci do N tříd).

Přidáním skrytých vrstev může síť vytvářet složité, nelineární rozhodovací hranice. Každá vrstva transformuje data z předchozí vrstvy, což umožňuje síti jako celku aproximovat téměř jakoukoliv spojitou funkci.

2.5 Role nelinearity: Úvod do aktivačních funkcí

Mohlo by se zdát, že stačí jednoduše naskládat vrstvy na sebe. Pokud by však všechny neurony v síti používaly pouze lineární operace (jako je vážený součet), celá síť by se dala matematicky zredukovat na jedinou lineární transformaci, bez ohledu na počet vrstev. Přidávání

vrstev by tak postrádalo smysl.

Klíčem, který odemyká sílu hlubokých sítí, jsou **nelineární aktivační funkce**. Tyto funkce nahrazují jednoduchou krokovou funkci perceptronu a vnášejí do modelu potřebnou nelinearitu.

Musí být také diferencovatelné, což je, jak uvidíme později, klíčové pro trénování sítě.

- **Sigmoid:** Funkce ve tvaru písmene "S", která "stlačí" jakoukoliv vstupní hodnotu do intervalu (0, 1). Byla historicky velmi populární, ale v hlubokých sítích trpí problémem mizejících gradientů (vanishing gradients), který zpomaluje učení. Její rovnice je $\sigma(x) = 1/(1 + e^{-x})$.
- **ReLU (Rectified Linear Unit):** Dnešní standardní a nejpoužívanější aktivační funkce. Její definice je velmi jednoduchá: $f(x) = \max(0, x)$. Pokud je vstup pozitivní, výstup je stejný; pokud je negativní, výstup je nula. Je výpočetně velmi efektivní a pomáhá řešit problém mizejících gradientů. Jejím neduhem může být problém "mrtvých neuronů" (dying ReLU), kdy neuron přestane být aktivní pro jakýkoli vstup.

Evoluce od jediného perceptronu k hlubokému MLP je příběhem postupného překonávání fundamentálních omezení přidáním dvou klíčových ingrediencí: **hloubky (skrytých vrstev)** a **nelinearity (aktivačních funkcí)**. Perceptron je lineární klasifikátor, který selhává na nelineárních problémech. Řešením je MLP , ale pouhé skládání lineárních vrstev je matematicky neúčinné. Nelineární aktivační funkce jsou tedy tou esenciální složkou, která činí skryté vrstvy efektivními. Každá vrstva nelineárně "pokřiví" příznakový prostor, což umožňuje následující vrstvě kreslit nové lineární hranice v tomto pokřiveném prostoru. Právě kompozice těchto nelineárních transformací umožňuje síti aproximovat libovolně složité funkce. Síla hluboké sítě tedy pramení z *interakce* mezi její hloubkou a nelinearitou.

Vlastnost	Sigmoid	ReLU (Rectified Linear Unit)
Vzorec	$\sigma(x) = 1/(1 + e^{-x})$	$\max(0, x)$
Výstupní rozsah	(0, 1)	$[0, \infty)$
Výhody	Hladký gradient, pravděpodobnostní výstup	Výpočetně efektivní, zabraňuje mizejícím gradientům
Nevýhody	Problém mizejících gradientů, výpočetně náročná	Problém "mrtvých neuronů", není centrovaná kolem nuly

3. Proces učení: Jak se síť trénuje

Nyní, když známe anatomii neuronové sítě, podívejme se na to, jak se "učí". Proces trénování je iterativní cyklus, jehož cílem je systematicky upravovat váhy a prahy sítě tak, aby co nejlépe plnila zadaný úkol. Tento proces lze rozdělit do tří základních kroků.

3.1 Cíl: Minimalizace chyby pomocí ztrátové funkce

Než začneme síť trénovat, musíme definovat, co znamená "dělat chyby". K tomu slouží

ztrátová funkce (Loss Function, také Cost nebo Error Function). Je to matematická funkce, která měří, jak moc se predikce sítě liší od skutečných, správných hodnot (ground truth) v trénovacích datech.
Cílem celého trénovacího procesu je najít takovou kombinaci vah a prahů v celé síti, která hodnotu této ztrátové funkce minimalizuje.

3.2 Krok 1: Průchod vpřed (Forward Pass) - Vytvoření predikce

Vše začíná **průchodem vpřed** (Forward Propagation). Vezmeme jeden vzorek (nebo dávku vzorků) z trénovacích dat a "proženeme" ho sítí od vstupní vrstvy až po výstupní.

1. Data vstoupí do vstupní vrstvy.
2. V každé následující vrstvě se pro každý neuron spočítá vážený součet vstupů z předchozí vrstvy.
3. Tento součet projde aktivační funkcí, jejíž výstup se stane vstupem pro další vrstvu.
4. Tento proces pokračuje, dokud výstupní vrstva nevygeneruje finální predikci.

Při prvním průchodu, kdy jsou váhy a prahy nastaveny náhodně, bude tato predikce samozřejmě zcela chybná.

3.3 Krok 2: Zpětný průchod (Backpropagation) - Přisouzení viny za chybu

Jakmile máme predikci a spočítali jsme chybu pomocí ztrátové funkce, musíme zjistit, jak každý jednotlivý parametr (váha a práh) v síti k této chybě přispěl. K tomu slouží algoritmus zvaný **zpětná propagace chyby** (Backpropagation).
Backpropagation funguje tak, že šíří signál o chybě zpětně, od výstupní vrstvy k vstupní. Pomocí řetězového pravidla z diferenciálního počtu vypočítá **gradient** ztrátové funkce vzhledem ke každému parametru v síti. Gradient je vektor, který pro každý parametr ukazuje směr nejstrmějšího růstu ztrátové funkce. Intuitivně nám říká: "Pokud tento parametr nepatrně zvýšíš, jak moc se změní celková chyba a jakým směrem?"

3.4 Krok 3: Optimalizace pomocí gradientního sestupu - Krok správným směrem

Když známe gradient, víme, jakým směrem se ztrátová funkce nejrychleji zvyšuje. Protože naším cílem je ztrátu *minimalizovat*, uděláme malý krok v **přesně opačném směru**, než ukazuje gradient. Tento postup se nazývá **gradientní sestup** (Gradient Descent).
Aktualizace každého parametru se provádí podle jednoduchého pravidla: $w_{nová} = w_{stará} - \alpha \cdot \nabla L$, kde w je parametr (váha nebo práh), α je **rychlost učení** (learning rate), která určuje velikost kroku, a ∇L je gradient ztrátové funkce L vzhledem k danému parametru.
Tento tříkrokový cyklus (Průchod vpřed → Výpočet ztráty → Zpětná propagace → Aktualizace vah) se opakuje mnohokrát pro všechny vzorky v trénovací sadě, dokud ztráta neklesne na

přijatelnou úroveň.

3.5 Analogie: Sestup z hory v mlze

Celý proces si lze představit pomocí následující analogie:

- **Hora:** Představuje vícerozměrný povrch ztrátové funkce. Každý bod na povrchu odpovídá určité kombinaci vah sítě.
- **Vaše poloha (zeměpisná šířka a délka):** Aktuální hodnoty vah a prahů ve vaší síti.
- **Vaše nadmořská výška:** Aktuální hodnota ztrátové funkce.
- **Cíl:** Dostat se do nejnižšího bodu v údolí (globální minimum ztráty).
- **Mlha:** Nevidíte celou krajinu. Můžete cítit pouze sklon svahu přímo pod vašima nohama.
- **Gradient:** Směr nejstrmějšího svahu v místě, kde stojíte.
- **Gradientní sestup:** Cítíte sklon a uděláte malý krok z kopce dolů.
- **Rychlost učení:** Velikost vašeho kroku. Pokud je příliš velká, můžete údolí "přeskočit". Pokud je příliš malá, bude vám sestup trvat věčně.

Celý proces učení v neuronové síti je v podstatě efektivní, iterativní vyhledávací algoritmus. Hluboká síť může mít miliony parametrů, což vytváří řešicí prostor s astronomickým počtem dimenzí. Najít v něm minimum hrubou silou je nemožné. Gradientní sestup poskytuje navigační princip: jdi vždy z kopce. Abychom však věděli, kterým směrem je "z kopce" pro každý jednotlivý parametr, potřebujeme gradient. Naivní výpočet milionů gradientů by byl výpočetně katastrofální. A právě zde se Backpropagation stává klíčovou technologií. Je to algoritmus, který tyto miliony gradientů vypočítá efektivně v jediném zpětném průchodu. Tím se "učení" demystifikuje a rámuje se jako inženýrský problém optimalizace a vyhledávání, jehož úspěch závisí jak na teoretické síle sítí, tak na algoritmické efektivitě a hardwarové akceleraci.

4. Praktický průvodce trénováním neuronových sítí

Teorie nám říká, *jak* trénování funguje. V praxi však musíme učinit řadu rozhodnutí, která zásadně ovlivní, *jak dobře* bude fungovat. Tato sekce se věnuje klíčovým volbám, které musí každý praktik učinit: výběru optimalizátoru, ladění hyperparametrů a boji proti přeučení.

4.1 Optimalizace sestupu: Výběr správné cesty z hory

Základní gradientní sestup má několik variant, které se liší v tom, kolik dat používají pro výpočet jednoho gradientu a aktualizaci vah.

- **Dávkový gradientní sestup (Batch GD):** Pro každou aktualizaci vah použije **celou** trénovací sadu. Výpočet gradientu je velmi přesný, což vede k hladké a stabilní konvergenci. Je však výpočetně extrémně náročný pro velké datasety a může uvíznout v lokálním minimu.

- **Stochastický gradientní sestup (Stochastic GD, SGD):** Pro každou aktualizaci použije pouze **jeden náhodně vybraný vzorek**. Aktualizace jsou velmi rychlé a časté. "Šum" v odhadu gradientu může pomoci uniknout z lokálních minim. Konvergenční cesta je však velmi nestabilní a chaotická.
- **Mini-dávkový gradientní sestup (Mini-Batch GD):** Praktický kompromis mezi oběma extrémy. Pro každou aktualizaci použije malou dávku (mini-batch) dat (typicky 32, 64, 128 vzorků). Tím kombinuje výhody obou přístupů: relativně stabilní konvergenci a efektivní využití hardwarové akcelerace (vektorizace na GPU). Je to de facto standard v moderním hlubokém učení.

Kromě volby velikosti dávky existují i chytřejší optimalizační algoritmy, které vylepšují základní gradientní sestup.

- **Momentum:** Tento přístup přidává k aktualizaci vah "setrvačnost". Představte si kuličku valící se z kopce – nejenže se řídí sklonem, ale má i svou hybnost. To pomáhá urychlit sestup v plochých oblastech a tlumit oscilace v úzkých a strmých údolích (tzv. patologické zakřivení).
- **Adam (Adaptive Moment Estimation):** Dnešní nejpoužívanější optimalizátor. Kombinuje myšlenku momentu s adaptivní rychlostí učení pro každý jednotlivý parametr. To znamená, že parametry, které mají konzistentní gradient, se učí rychleji, zatímco u parametrů s oscilujícím gradientem se rychlost učení snižuje. Pro většinu problémů je Adam robustní a efektivní volbou, která často nevyžaduje složité ladění.

Aspekt	Dávkový GD (Batch)	Stochastický GD (SGD)	Mini-dávkový GD (Mini-Batch)
Data na aktualizaci	Celý dataset	Jeden vzorek	Dávka n vzorků
Frekvence aktualizací	Velmi nízká (1 za epochu)	Velmi vysoká (N za epochu)	Střední (N/n za epochu)
Konvergenční cesta	Hladká, stabilní	Šumivá, nestabilní	Mírně šumivá
Výpočetní náročnost	Velmi vysoká	Nízká	Střední (optimalizováno pro GPU)

4.2 Ladění motoru: Základní hyperparametry

Zatímco **parametry** sítě (váhy a prahy) se model učí sám, **hyperparametry** jsou vnější nastavení trénovacího procesu, která musíme zvolit my, praktici.

- **Rychlost učení (Learning Rate, α):** Pravděpodobně nejdůležitější hyperparametr. Řídí velikost kroku při gradientním sestupu. Pokud je příliš vysoká, trénink může "přestřelovat" minimum a nikdy nekonvergovat. Pokud je příliš nízká, trénink bude extrémně pomalý.
- **Velikost dávky (Batch Size):** Počet vzorků v jedné mini-dávce. Ovlivňuje rychlost tréninku, využití paměti a "šumivost" gradientu. Větší dávky vedou ke stabilnějším gradientům, ale vyžadují více paměti a mohou vést k horší generalizaci.
- **Počet epoch (Epochs):** Jedna epocha znamená jeden kompletní průchod celé trénovací sady dat sítí. Počet epoch určuje, jak dlouho se model bude trénovat.

4.3 Problém přeučení: Když se model učí až příliš dobře

Nejčastějším problémem v praxi je **přeučení (overfitting)**. K tomu dochází, když má model vynikající výsledky na trénovacích datech, ale špatně si vede na nových, neviděných datech (validační nebo testovací sada). Model se v podstatě *naučil nazpaměť* trénovací data, včetně jejich šumu a specifik, místo aby se naučil obecný, podkladový vzor.

Diagnostika přeučení: Typickým příznakem je, když trénovací ztráta stále klesá, zatímco validační ztráta začne stagnovat nebo dokonce stoupat.

Regularizační techniky: K boji proti přeučení slouží regularizační techniky, které do trénovacího procesu vnášejí určitou formu "penalizace" za přílišnou složitost modelu.

- **L1/L2 regularizace:** Přidává do ztrátové funkce penále založené na velikosti vah. Tím "nutí" model, aby preferoval menší váhy, což vede k jednodušším a robustnějším modelům.
- **Dropout:** Během tréninku se v každém kroku náhodně "vypne" (nastaví na nulu) určitá část neuronů v dané vrstvě. To nutí síť, aby se nespoléhala na několik málo specifických neuronů a učila se redundantní, robustnější reprezentace.
- **Předčasné ukončení (Early Stopping):** Během tréninku se sleduje výkon na validační sadě. Jakmile se validační ztráta přestane zlepšovat (nebo se začne zhoršovat) po určitý počet epoch, trénink se zastaví. Tím se zabrání tomu, aby se model začal přeučovat.

Trénování neuronové sítě není jednorázový proces, ale empirická věda o hledání rovnováhy v tzv. **kompromisu mezi vychýlením a rozptylem (bias-variance trade-off)**. Nástroje, které pomáhají modelu učit se rychleji a lépe se přizpůsobit trénovacím datům (vysoká rychlost učení, velká kapacita modelu), zároveň zvyšují riziko přeučení (vysoký rozptyl). Naopak techniky, které zabraňují přeučení (silná regularizace), mohou modelu bránit v efektivním učení podkladového vzoru, což vede k podučení (vysoké vychýlení). Úkolem praktika je navigovat tento kompromis. Například pokud zvětšíte model (snížíte vychýlení), pravděpodobně budete muset zvýšit i regularizaci (např. míru dropoutu), abyste kontrolovali výsledný nárůst rozptylu.

Technika	Mechanismus	Primární efekt
L1/L2 regularizace	Přidává penalizaci za velikost vah do ztrátové funkce.	Podporuje menší hodnoty vah, což vede k jednoduššímu modelu.
Dropout	Náhodně deaktivuje neurony během tréninku.	Zabraňuje ko-adaptaci neuronů a zvyšuje robustnost modelu.
Předčasné ukončení	Sleduje validační ztrátu a zastaví trénink, když začne stoupat.	Zabraňuje příliš dlouhému tréninku a memorování šumu.

5. Moderní pracovní postup a ekosystém hlubokého učení

Tato krátká sekce představuje praktické nástroje a myšlenkové postupy nezbytné pro zahájení vývoje a ladění vlastních modelů.

5.1 Rychlý přehled frameworků: PyTorch vs. TensorFlow

V současnosti dominují dva hlavní frameworky pro hluboké učení: **PyTorch** (vyvíjený společností Meta) a **TensorFlow** (vyvíjený společností Google).

- **PyTorch** je často vnímán jako více "pythonický", flexibilnější a je dominantní ve výzkumné komunitě. Jeho dynamická povaha usnadňuje ladění a experimentování.
- **TensorFlow** má historicky silnější pozici v produkčním nasazení díky robustním nástrojům jako TensorFlow Serving a TensorFlow Lite pro mobilní zařízení.

Pro začátečníka je důležité vědět, že základní koncepty jsou přenositelné mezi oběma frameworky a znalost jednoho z nich je skvělým výchozím bodem.

5.2 Kritická kontrola příčetnosti: Umění přeučit jednu dávku

Než spustíte trénink na plném datasetu, který může trvat hodiny nebo dny, je naprosto klíčové provést jeden jednoduchý, ale nesmírně účinný test: **pokusit se přeučit jednu jedinou mini-dávku dat**.

Postup:

1. Vezměte jednu malou mini-dávku (např. 32 vzorků).
2. Vypněte veškerou regularizaci (dropout, L1/L2 atd.).
3. Trénujte model na této jediné dávce po mnoho epoch.

Cíl: Dosáhnout na této dávce téměř nulové trénovací ztráty.

Proč je to důležité? Pokud váš model nedokáže ani dokonale "zapamatovat" malé množství dat, znamená to, že je v něm fundamentální chyba. Může jít o chybu v architektuře modelu, v datovém kanálu, v implementaci ztrátové funkce nebo ve zpětné propagaci. Tento test vám dá jistotu, že model má dostatečnou kapacitu a že celý učící mechanismus funguje správně, než ztratíte čas a výpočetní zdroje na plnohodnotném tréninku.

5.3 Monitorování tréninku: Vizualizace pokroku

Trénování hlubokých modelů není proces, který "nastavíte a zapomenete". Je klíčové sledovat jeho průběh v reálném čase, abyste mohli diagnostikovat problémy. K tomu slouží nástroje pro vizualizaci a experimentální sledování, jako jsou **TensorBoard** (nativní pro TensorFlow, ale kompatibilní i s PyTorch) a **Weights & Biases (W&B)**. Tyto platformy umožňují logovat a vykreslovat klíčové metriky, jako je ztráta, přesnost, rychlost učení, a také sledovat využití

systémových zdrojů (CPU/GPU).

Praktická aplikace hlubokého učení je silně závislá na robustním ekosystému softwarových frameworků a nástrojů (MLOps). Schopnost efektivně ladit a systematicky monitorovat experimenty je stejně důležitá jako porozumění základní teorii. Úspěch v aplikovaném DL pochází z disciplinovaného cyklu implementace, kontroly příčetnosti, systematického experimentování a ladění.

6. Pohled vpřed: Proč MLP pro materiálovou vědu nestačí

Tato závěrečná sekce je klíčová pro narativní oblouk celého kurzu. Explicitně se zabývá omezeními "klasického" vícevrstvého perceptronu pro běžné typy dat v materiálové vědě a přímo tak motivuje potřebu specializovaných architektur, kterým se budeme věnovat v následujících přednáškách.

6.1 Výzva prostorových dat: Snímky mikrostruktur

- **Problém:** MLP vyžaduje, aby byl 2D obrázek "zploštěn" do 1D vektoru. Tento proces zcela zničí veškeré prostorové vztahy mezi pixely. Síť nemá žádnou vrozenou informaci o tom, že pixely, které jsou si blízko, spolu souvisejí více než pixely na opačných koncích obrázku.
- **Důsledky:**
    - **Exploze parametrů:** Plně propojená vrstva pro středně velký obrázek vede k obrovskému počtu parametrů (vah), což činí model náchylným k přeučení a pomalým na trénování. Například pro obrázek 224x224 pixelů ve 3 barevných kanálech by vstupní vrstva měla zhruba 150 000 neuronů.
    - **Chybějící translační invariance:** Síť se učí příznaky vázané na konkrétní pozice. Pokud se hranice zrna objeví v levém horním rohu jednoho trénovacího snímku, model nerozpozná stejný vzor, pokud se objeví v pravém dolním rohu nového snímku, aniž by viděl obrovské množství dat se všemi možnými posuny.

- **Most k další přednášce:** Toto selhání přímo motivuje potřebu **Konvolučních neuronových sítí (CNN)**, které jsou speciálně navrženy s tzv. induktivními předsudky (inductive biases) – lokální konektivitou a sdílením vah – pro efektivní zpracování prostorových dat.

6.2 Výzva relačních dat: Krystalové struktury

- **Problém:** Krystalová struktura není jen seznam atomů; je to **graf** definovaný atomy (uzly) a jejich vazbami (hranami) v periodické mřížce. MLP nedokáže přirozeně reprezentovat tuto konektivitu ani periodicitu. Bylo by možné vytvořit pro každý krystal vektor příznaků (např. průměrná délka vazby, koordinační číslo), ale tím se vracíme k manuální přípravě příznaků a ztrácíme bohaté relační informace.
- **Důsledky:** Model by neměl žádné povědomí o vazebném prostředí atomu, o tom, které atomy jsou sousedé, nebo o periodické povaze krystalu – což jsou všechno klíčové informace pro predikci materiálových vlastností.

- **Most k další přednášce:** Toto omezení motivuje potřebu **Grafových neuronových sítí (GNN)**, které operují přímo na grafových strukturách. Učí se reprezentace předáváním "zpráv" mezi propojenými uzly (atomy), čímž přirozeně zachycují lokální atomární prostředí. To navazuje na rostoucí využití GNN pro predikci vlastností krystalických materiálů.

6.3 Výzva sekvenčních dat: Spektroskopie a časové řady

- **Problém:** Data z technik jako je spektroskopie (např. XRD, Raman) nebo z časově závislých simulací jsou **sekvenční**. Hodnota v jednom bodě (např. při určité vlnové délce) je silně závislá na hodnotách, které jí předcházely. Standardní MLP předpokládá, že všechny vstupy jsou na sobě nezávislé a nemá žádný koncept pořadí nebo času.
- **Důsledky:** Model nedokáže zachytit časové závislosti nebo dlouhodobé vzory v sekvenci. Každý datový bod zpracovává izolovaně.

- **Most k další přednášce:** To motivuje potřebu architektur s "pamětí", jako jsou **Rekurentní neuronové sítě (RNN)** a v poslední době především **Transformery**, které používají mechanismy pozornosti k modelování závislostí na dlouhé vzdálenosti v sekvencích. To souvisí s použitím těchto modelů pro analýzu spektrálních dat nebo pro zpracování vědecké literatury.

Progrese od MLP ke specializovaným architekturám (CNN, GNN, Transformery) je příběhem o znovuzavádění doménových znalostí ve formě **induktivních předsudků**. "Klasický" MLP je mocný, protože je to univerzální aproximátor funkcí, ale je neefektivní, protože postrádá jakékoli předchozí předpoklady o struktuře dat. Materiálovědná data jsou však téměř vždy vysoce strukturovaná. Selhání MLP na těchto úlohách pramení přímo z tohoto nesouladu. Specializované architektury jako CNN a GNN jsou úspěšné právě proto, že tyto předpoklady zabudovávají přímo do své architektury. Konvoluční filtr v CNN je induktivní předsudek pro lokalitu a translační invarianci. Mechanismus předávání zpráv v GNN je induktivní předsudek pro relační strukturu. Volba specializované architektury je tedy vědomým aktem, kdy se vzdáváme části univerzálnosti výměnou za model, který je mnohem efektivnější a účinnější pro specifický typ dat a fyzikální problém.

7. Závěr a zdroje pro hlubší studium

Tato přednáška položila základy pro pochopení neuronových sítí a hlubokého učení. Prošli jsme si cestu od historického kontextu a základní anatomie až po praktické aspekty trénování a klíčová omezení, která nás povedou v dalších částech kurzu.

7.1 Shrnutí klíčových konceptů

- Hluboké učení se od tradičního strojového učení liší především **automatickým učením reprezentací** z hrubých dat, což eliminuje potřebu manuální přípravy příznaků.
- Základním stavebním kamenem je **neuron** (perceptron), a jejich skládáním do vrstev vzniká **vícevrstvý perceptron (MLP)**, který díky **nelineárním aktivačním funkcím** dokáže modelovat složité vztahy.
- Trénování probíhá v cyklu **průchodu vpřed**, výpočtu **ztráty**, **zpětné propagace** chyby a aktualizace vah pomocí **gradientního sestupu**.
- V praxi je klíčové správně nastavit **hyperparametry** (rychlost učení, velikost dávky) a používat **regularizační techniky** (Dropout, L1/L2) k boji proti **přeučení**.
- Standardní MLP mají zásadní omezení pro zpracování strukturovaných dat, jako jsou obrázky, grafy nebo sekvence, což motivuje použití specializovaných architektur (CNN, GNN, Transformery).

7.2 Interaktivní učení: Demonstrace TensorFlow Playground

Pro budování intuice a hravé experimentování s koncepty, které jsme dnes probrali, neexistuje lepší nástroj než **TensorFlow Playground**. Jedná se o webovou aplikaci, která běží přímo ve vašem prohlížeči a umožňuje vám:

- Vizuálně sestavovat jednoduché neuronové sítě, měnit počet vrstev a neuronů.
- Experimentovat s různými datasety (lineárně separabilními i nelineárními).
- Měnit hyperparametry jako rychlost učení, aktivační funkci nebo regularizaci a v reálném čase sledovat, jak se mění rozhodovací hranice a ztrátová funkce.

Tato "laboratoř v prohlížeči" je neocenitelným nástrojem pro pochopení toho, jak jednotlivé komponenty ovlivňují proces učení.

V příští přednášce se zaměříme na první z našich specializovaných architektur: Konvoluční neuronové sítě a jejich aplikaci v analýze mikrostruktur a dalších obrazových dat v materiálovém inženýrství.

Works cited

1. A CNN-Based Method for Quantitative Assessment of Steel Microstructures in Welded Zones, https://www.mdpi.com/2079-6439/13/5/66 2. Classification of Steel Microstructure Image Using CNN - ResearchGate, https://www.researchgate.net/publication/377039577_Classification_of_Steel_Microstructure_Image_Using_CNN 3. A review on the applications of graph neural networks in materials science at the atomic scale - ResearchGate, https://www.researchgate.net/publication/381324535_A_review_on_the_applications_of_graph_neural_networks_in_materials_science_at_the_atomic_scale 4. Examining graph neural networks for crystal structures: Limitations and opportunities for capturing periodicity - DSpace@MIT, https://dspace.mit.edu/handle/1721.1/154281 5. SpecTf: Transformers enable data-driven imaging spectroscopy cloud detection | PNAS, https://www.pnas.org/doi/10.1073/pnas.2502903122 6. [Literature Review] Materials science in the era of large language models: a perspective, https://www.themoonlight.io/en/review/materials-science-in-the-era-of-large-language-models-a-perspective 7. Difference Between AI, ML, DL Neural Networks, and how ChatGPT ..., https://loqos.ai/blog/difference_between_ai_ml_dl_nn 8. Understanding the Hierarchy of Artificial Intelligence (AI) - RPATech, https://www.rpatech.ai/hierarchy-of-ai/ 9. 1.AI vs ML vs DL: Understanding the Hierarchy of Intelligent Technologies | by Kiran vutukuri, https://medium.com/@kiranvutukuri/1-ai-vs-ml-vs-dl-understanding-the-hierarchy-of-intelligent-technologies-5126c29e6c64 10. Comparing Deep Learning and Traditional Machine Learning - The CEO Views, https://theceoviews.com/comparing-deep-learning-and-traditional-machine-learning/ 11. What are the key differences between traditional machine learning ..., https://eitca.org/artificial-intelligence/eitc-ai-adl-advanced-deep-learning/introduction-eitc-ai-adl-advanced-deep-learning/introduction-to-advanced-machine-learning-approaches/examination-review-introduction-to-advanced-machine-learning-approaches/what-are-the-key-differences-between-traditional-machine-learning-and-deep-learning-particularly-in-terms-of-feature-engineering-and-data-representation/ 12. Perceptron - Wikipedia, https://en.wikipedia.org/wiki/Perceptron 13. What is a Perceptron: Components, Characteristics, and Types, https://www.simplilearn.com/tutorials/deep-learning-tutorial/perceptron 14. Lecture 5: Multilayer Perceptrons, https://www.cs.toronto.edu/~rgrosse/courses/csc321_2018/readings/L05%20Multilayer%20Perceptrons.pdf 15. Multilayer perceptron - Wikipedia, https://en.wikipedia.org/wiki/Multilayer_perceptron 16. Understanding Activation Functions: ReLU vs Sigmoid | by Shubh ..., https://medium.com/@shuklashubh818/understanding-activation-functions-relu-vs-sigmoid-0b24fd636ff9 17. Intuitive understanding of forward and backward propagation - DeepLearning.AI, https://community.deeplearning.ai/t/intuitive-understanding-of-forward-and-backward-propagation/593917 18. What is Backpropagation? | IBM, https://www.ibm.com/think/topics/backpropagation 19. Multi-Layer Perceptron Learning in Tensorflow - GeeksforGeeks, https://www.geeksforgeeks.org/deep-learning/multi-layer-perceptron-learning-in-tensorflow/ 20. Gradient Descent : Batch , Stocastic and Mini batch | by Aman ..., https://medium.com/@amannagrawall002/batch-vs-stochastic-vs-mini-batch-gradient-descent-techniques-7dfe6f963a6f 21. Difference between Batch Gradient Descent and Stochastic Gradient Descent - GeeksforGeeks,

https://www.geeksforgeeks.org/machine-learning/difference-between-batch-gradient-descent-and-stochastic-gradient-descent/ 22. Intuitive Explanation of SGD, Adam, and RMSProp - Kaggle, https://www.kaggle.com/code/harpdeci/intuitive-explanation-of-sgd-adam-and-rmsprop 23. Intro to optimization in deep learning: Momentum, RMSProp and ..., https://www.digitalocean.com/community/tutorials/intro-to-optimization-momentum-rmsprop-adam 24. Linear regression: Hyperparameters | Machine Learning - Google for Developers, https://developers.google.com/machine-learning/crash-course/linear-regression/hyperparameters 25. What Does Batch Size Mean in Deep Learning? An In-Depth Guide ..., https://www.coursera.org/articles/what-does-batch-size-mean-in-deep-learning 26. Neural Network Regularization Techniques | Coursera, https://www.coursera.org/articles/neural-network-regularization 27. Mastering Dropout in Neural Networks: A Comprehensive Guide to Preventing Overfitting, https://www.lunartech.ai/blog/mastering-dropout-in-neural-networks-a-comprehensive-guide-to-preventing-overfitting 28. Lecture 7: Troubleshooting Deep Neural Networks - The Full Stack, https://fullstackdeeplearning.com/spring2021/lecture-7/ 29. TensorFlow vs PyTorch: A Comparative Analysis for 2025 | Leapcell, https://leapcell.io/blog/tensorflow-vs-pytorch-a-comparative-analysis-for-2025 30. Is PyTorch better than TensorFlow for beginners in AI/ML in 2025?, https://dev-discuss.pytorch.org/t/is-pytorch-better-than-tensorflow-for-beginners-in-ai-ml-in-2025/3033 31. What's your go-to approach for debugging a neural network that isn't training as expected? : r/deeplearning - Reddit, https://www.reddit.com/r/deeplearning/comments/1guj7ax/whats_your_goto_approach_for_debugging_a_neural/ 32. TensorBoard - Weights & Biases Documentation - Wandb, https://docs.wandb.ai/guides/integrations/tensorboard/ 33. Weights & Biases: The AI Developer Platform, https://wandb.ai/site/ 34. TensorBoard - Weights & Biases Documentation, https://docs.wandb.ai/guides/integrations/tensorboard 35. CNN vs MLP for Image Classification | by Dinesh | Analytics Vidhya - Medium, https://medium.com/analytics-vidhya/cnn-convolutional-neural-network-8d0a292b4498 36. Shorts: Why we can't use a Multi-Layer Perceptron for object detection - Medium, https://medium.com/mlearning-ai/shorts-why-we-cant-use-a-multi-layer-perceptron-for-object-detection-3d158220f69f 37. The Problem with Traditional Neural Networks (MLP's) - Kaggle, https://www.kaggle.com/discussions/general/511294 38. Advanced Steel Microstructural Classification by Deep Learning Methods - PMC - NIH, https://pmc.ncbi.nlm.nih.gov/articles/PMC5794926/ 39. Examining graph neural networks for crystal structures: Limitations and opportunities for capturing periodicity - PMC - NIH, https://pmc.ncbi.nlm.nih.gov/articles/PMC10637739/ 40. Crystal graph attention networks for the prediction of stable materials - PMC - NIH, https://pmc.ncbi.nlm.nih.gov/articles/PMC8641929/ 41. Sequential Data — and the Neural Network Conundrum! | by Aashish Chaubey | Analytics Vidhya | Medium, https://medium.com/analytics-vidhya/sequential-data-and-the-neural-network-conundrum-b2c005f8f865 42. Recurrent neural network - Wikipedia, https://en.wikipedia.org/wiki/Recurrent_neural_network 43. Evaluation of DNN and RNN for the determination of the chemical composition of dimensionality-reduced LIBS spectra of fusion-based materials | Request PDF - ResearchGate, https://www.researchgate.net/publication/395934227_Evaluation_of_DNN_and_RNN_for_the_determination_of_the_chemical_composition_of_dimensionality-reduced_LIBS_spectra_of_fusion-based_materials 44. artificial intelligence in spectroscopy: advancing chemistry from prediction to generation and beyond - arXiv, https://arxiv.org/pdf/2502.09897? 45. Full article:

MaterialBERT for natural language processing of materials science texts, https://www.tandfonline.com/doi/full/10.1080/27660400.2022.2124831 46. Are multilayer perceptron models still usable in the industry today? : r/learnmachinelearning, https://www.reddit.com/r/learnmachinelearning/comments/1k17tsg/are_multilayer_perceptron_models_still_usable_in/ 47. TensorFlow Playground: Making Deep Learning Easy - DataScientest, https://datascientest.com/en/all-about-deep-learning-with-tensorflow-playground 48. TensorFlow Playground : A walkthrough - GeeksforGeeks, https://www.geeksforgeeks.org/deep-learning/tensorflow-playground-a-walkthrough/ 49. Andrew Ng's Machine Learning Collection - Coursera, https://www.coursera.org/collections/machine-learning 50. Stanford University CS231n: Deep Learning for Computer Vision, https://cs231n.stanford.edu/ 51. CS231n Deep Learning for Computer Vision, https://cs231n.github.io/